Зміст

1 Синтаксис Python. Типи Даних 2

1.1 Змінні та Ідентифікатори 2

1.1.1 Правила побудови ідентифікаторів 2

1.2 Зарезервовані слова Python 2

1.2.1 Приклад помилки 3

1.3 Оператори та Вирази 3

1.3.1 Правила форматування коду 4

1.4 Основні Типи Даних і Операції 4

1.5 Приклади. Виділення цифр числа 6

1.5.1 Двоцифрове число 6

1.5.2 Трицифрове число 6

1.6 Вбудовані функції для роботи з числами 7

1.6.1 Функція `abs(a)` 7

1.6.2 Функція `divmod(a, b)` 7

1.6.3 Функція `hex(x)` 7

1.6.4 Функція `oct(x)` 7

1.6.5 Функція `round(x)` 8

1.6.6 Функція `round(x, n)` 8

2 Умовний оператор if 10

2.1 Оператори порівнювання 10

2.2 Синтаксичні форми умовного оператора 11

Синтаксис Python. Типи Даних

1

1.1 Змінні та Ідентифікатори

Змінна - це

ім’я (ідентифікатор), яке може набувати певного значення.

У Python змінні не потрібно оголошувати, значення можна надати одразу за допомогою символу =.

Приклади присвоєння

```
x = 4
s1 = "Hello"
_x = True
x2 = 23.46
summa1 = summa2 = k = 0
```

Ідентифікатор - це

унікальна ознака об’єкта, що дозволяє розрізняти його з іншими.

Правила побудови ідентифікаторів

- ☑ Використовуються лише латинські літери, цифри та знак підкреслення (_).
- ☑ Ідентифікатор повинен починатися з літери або символу підкреслення.
- ☑ Регістр літер має значення (`X` ≠ `x`).
- ☑ Неприпустимі пробіли, точки, спеціальні символи та літери кирилиці.
- ☑ Не можна застосовувати спеціальні символи `$`, `%`, `@`.
- ☑ Заборонено використовувати зарезервовані слова Python (`if`, `for`, `while`, `class`, `def` тощо).

1.2 Зарезервовані слова Python

У Python існує набір **ключових слів**, які не можна використовувати як ідентифікатори (назви змінних, функцій, класів тощо). Ці слова мають спеціальне значення

у синтаксисі мови.

```
False   class     finally  is       return
None    continue  for      lambda   try
True    def       from     nonlocal while
and     del       global   not      with
as      elif      if       or       yield
assert  else      import   pass     break
```

Використання будь-якого з цих слів як назви змінної або функції призведе до помилки.

Приклад помилки

```
>>> class = 5
File "<stdin>", line 1
class = 5
^
SyntaxError: invalid syntax
```

1.3 Оператори та Вирази

Вираз — це

фрагмент мови програмування, що представляє спосіб обчислення деякого значення.

Операнд — це

частина виразу, над яким виконуються операції.

Оператор — це

дія, яка може бути виконана над одним або декількома операндами.

Приклади синтаксису

```
X=4+5 % Присвоєння
Print(x)
S="Hello "+"World"
Print(s)
S="Helo"+\
"World" % Перенесення довгих операторів
```

Правила форматування коду

- ☑ Кожен оператор завершується символом нового рядка.
- ☑ Відступи застосовуються для позначення різних блоків коду (тіла функцій, умовні висловлювання, цикли).
- ☑ Пробіли неприпустимі перед операторами, але необхідні перед вкладеними блоками (`unexpected indent`).
- ☑ Якщо потрібно розмістити кілька операторів в одному рядку, їх розділяють крапкою з комою (`;`).
- ☑ Символ # означає коментар.
- ☑ Багаторядковий коментар укладають у три подвійні лапки в `py` файлі.

1.4 Основні Типи Даних і Операції

Вбудовані типи даних:

- ☑ Числа: `1234`, `3.14`, `3+4j`.
- ☑ Рядки: `'spam'`, `"guido's"`.
- ☑ Списки: `[1, [2, 'three'], 4]`.
- ☑ Словники: `{'food': 'spam', 'taste': 'yum'}`.
- ☑ Кортежі: `(1, 'span', 4, 'U')`.
- ☑ Файли: myfile = open(’eggs’, ‘r’)
- ☑ Множини: set(’abc’). ’a’,’b’, ’c’

Вбудовані числові типи

- ☑ цілі числа (числа без дробової частини), (3, 3465, 45, 42, -4) (з необмеженою точністю)
- ☑ дійсні числа (грубо кажучи, числа з десятковою точкою з плаваючою точкою) (3,15 5,68)

- ☑ 8-річні, починаються нуль про 0o77 = 63 0o10 = 8
- ☑ 16-річні, починаються нуль x 0xFF=255 0xff=255 • - двійкові, починаються нуль b 0b1111 = 15 0b10 = 2
- ☑ - комплексні числа з уявною частиною, (3+4j), (3+4J)(з допомогою великої та маленької) літер)

Символ множення перед j не вказується.

```
>>>z1=2+3j
>>>z2=4+8J
>>>print(z1, z2)
(2+3j) (4+8j)
```

Комплексні числа можуть бути створені за допомогою функції complex(реч _ частина, уявна _ частина)

```
>>>z3=complex(34,2)
>>>print(z3)
(34+2j)
```

Операції з числовими типами

Оператор	Опис	Приклад	Результат
+	Додавання	5+7	12
-	Віднімання	55-10	45
*	Множення	4*6	24
/	Ділення	7 / 2	3.5
//	Цілочисельне ділення	7 // 2	3
%	Остача від ділення	7 % 3	1
**	Піднесення до степеня	3 ** 4	81

Результатом операції буде новий об’єкт.

2 ● ● 100#2100

1267650600228229401496703205376

Зверніть увагу на результат останньої операції: у Python 3 цілі числа автоматично забезпечують **необмежену** точність. Наприклад, ви можете спробувати обчислити 2 ступенем 1 000 000 (але навряд чи варто це робити, оскільки на екран буде виведено число довжиною понад 300 000 знаків, що може зайняти тривалий

час!).

> len(str(2 ** 1000000))

Скільки цифр у дійсно великому числі

301030

1.5 Приклади. Виділення цифр числа

Двоцифрове число

Нехай маємо число `n = 47`. Щоб виділити окремі цифри:

Двоцифрове число

```
n = 47
d1 = n // 10    # перша цифра (десятки)
d2 = n % 10     # друга цифра (одиниці)
print("Десятки =", d1, "Одиниці =", d2)
```

Результат:

```
Десятки = 4 Одиниці = 7
```

Трицифрове число

Нехай маємо число `m = 583`. Щоб виділити окремі цифри:

Трицифрове число

```
m = 583
sot = m // 100          # сотні
des = (m // 10) % 10    # десятки
od = m % 10             # одиниці
print("Сотні =", sot, "Десятки =", des, "Одиниці =", od)
```

Результат:

```
Сотні = 5 Десятки = 8 Одиниці = 3
```

Пріоритет операторів - порядок виконання тут

1.6 Вбудовані функції для роботи з числами

Python надає низку **вбудованих функцій**, які можна застосовувати до числових типів.

Функція `abs(a)`

Повертає модуль числа.

Приклад

```
x = -12
print(abs(x))    # 12
```

Функція `divmod(a, b)`

Виконує ділення `a` на `b` та повертає пару чисел: частку і залишок.

Приклад

```
x = 23
y = 7
print(divmod(x, y))    # (3, 2)
```

Функція `hex(x)`

Конвертує ціле число у рядок із шістнадцяткових цифр.

Приклад

```
x = 15
print(hex(x))    # '0xf'
```

Функція `oct(x)`

Конвертує ціле число у рядок із вісімкових цифр.

Приклад

```
x = 15
print(oct(x))    # '0o17'
```

Функція `round(x)`

Округлює число до найближчого цілого.

Приклад

```
print(round(3.0254))    # 3
```

Функція `round(x, n)`

Округлює число `x` з точністю до `n` знаків після коми.

Приклад

```
print(round(3.0254, 2))    # 3.03
```

Контрольні питання до глави 3

1. Що таке змінна у Python? Як їй надається значення?
2. Що таке ідентифікатор? Які правила його побудови?
3. Які символи та слова заборонено використовувати як ідентифікатори?
4. Що таке зарезервовані слова у Python? Наведіть приклади.
5. Що станеться, якщо використати зарезервоване слово як назву змінної?
6. Дайте означення виразу, операнда та оператора.
7. Які правила форматування коду у Python ви знаєте?
8. Як у Python позначають коментарі? Чим відрізняється однорядковий від багаторядкового коментаря?
9. Назвіть основні вбудовані типи даних у Python.
10. Які числові типи даних підтримує Python?
11. Як у Python записуються цілі, дійсні, восьмирічні, шістнадцятирічні та двійкові числа?
12. Як створюються комплексні числа у Python? Наведіть приклад.
13. Які основні арифметичні оператори підтримує Python? Наведіть приклади їх використання.
14. Що означає оператор цілочисельного ділення та оператор остачі від ділення?
15. Які вбудовані функції для роботи з числами ви знаєте? Наведіть приклади їх використання.

16. Як працює функція `round()`? Чим відрізняється `round(x)` від `round(x, n)`?
17. Як можна виділити окремі цифри з двоцифрового чи трицифрового числа?

Практичні завдання до глави 3

1. Створіть змінну `x` та надайте їй значення 25. Виведіть її на екран.
2. Складіть програму, яка перевіряє, чи є число парним або непарним.
3. Напишіть програму, яка виділяє десятки та одиниці з двоцифрового числа.
4. Напишіть програму, яка виділяє сотні, десятки та одиниці з трицифрового числа.
5. Обчисліть результат виразу: `(15 // 4) + (15 % 4)`. Поясніть отриманий результат.
6. Створіть дві змінні `z1` та `z2` як комплексні числа. Виконайте їх додавання та виведіть результат.
7. Використайте функцію `abs()` для знаходження модуля від'ємного числа.
8. Використайте функцію `divmod()` для ділення числа 29 на 5. Виведіть частку та остачу.
9. Використайте функцію `round()` для округлення числа 3.14159 до двох знаків після коми.
10. Напишіть програму, яка обчислює кількість цифр у числі 2^{1000}.

Умовний оператор if

2

За замовчуванням хід виконання програми є лінійним: інструкції виконуються послідовно — рядок за рядком, від першого до останнього. Послідовне виконання є базою, але реальні алгоритми вимагають гнучкості та адаптивності.

У більшості задач програма повинна приймати рішення залежно від вхідних даних чи поточного стану обчислень. Для цього використовуються розгалуження (або умовні переходи).

Оператор if в Python — це

умовний оператор, який дозволяє програмі приймати рішення та виконувати певний блок коду лише тоді, коли виконується задана умова

В основі прийняття рішень у програмі лежать оператори порівняння (відношення). Вони дозволяють зіставляти значення виразів і повертають результатом булеве значення — **True (істина)** або **False (хибність)**.

2.1 Оператори порівнювання

Python підтримує базові математичні операції відношення:

```
Дорівнює:                a == b
Не дорівнює:             a != b
Менше ніж:               a < b
Менше або дорівнює:     a <= b
Більше ніж:              a > b
Більше або дорівнює:    a >= b
```

Зверніть увагу:

Оператор перевірки на **рівність** позначено двома знаками ==, оскільки один знак = в Python зарезервовано для операції **присвоєння**.

На відміну від багатьох інших мов програмування, Python дозволяє використовувати ланцюгові порівняння (подвійні нерівності), що звично для математиків, наприклад: **a < x <= b** замість запису через логічне «і».

Отримані булеві вирази є фундаментом для керування потоком виконання про-

грами: вони використовуються в умовних конструкціях (if, elif, else) та циклах (while).

2.2 Синтаксичні форми умовного оператора

Залежно від кількості напрямків (гілок) виконання програми, в Python застосовують три основні форми розгалуження:

1. Неповна форма (коротке розгалуження)

Використовується, коли певну дію потрібно виконати **тільки** за умови, що вираз-умова є істинним (`True`). Якщо умова хибна (`False`), блок коду просто пропускається.

```
Неповна форма
if вираз_умова1:
    оператор1
    оператор2
```

2. Повна форма (двохгілкове розгалуження)

Застосовується, коли є два альтернативних шляхи: один виконується при істинності умови (`True`), а інший (блок `else`) — якщо умова хибна (`False`).

```
Повна форма
if вираз_умова1:
    оператор1
    оператор2
else:
    Блок_операторів_3
```

3. Багатогілкова форма (багатоальтернативне розгалуження)

Використовується для перевірки серії послідовних умов за допомогою конструкцій `elif` (скорочення від *else if*). Умови перевіряються зверху вниз: виконується блок першої умови, яка виявиться істинною, після чого весь оператор `if` завершує роботу.

```
Багатогілкова форма
if вираз_умова1:
    Блок_операторів_1
elif вираз_умова2:
    Блок_операторів_2
elif вираз_умова_n:
    Блок_операторів_n
else:
   Блок_операторів_n+1
```

Важливі синтаксичні особливості Python:

☑ **Двокрапка (:):** є обов'язковою в кінці кожного рядка з `if`, `elif` та `else`. Вона позначає початок відповідного блоку коду.

Приклад 1. Користувач вводить число та отримує відповідь яке воно позитивне негативне або нуль.

```
x=int(input("Введіть x "))
if x>0:
    print("x позитивне")
else:
    print("x негативне або нуль")
    input()
```

Приклад 2. Користувач вводить два числа та отримує відповідь яке більше або дорівнюють.

```
A=input()
B=input()
if A>B: print("A більше") #можно писати у тому ж рядку один оп-р
elif A==B: print("A дорівнює B")
else: print("A менше B")
```

Приклад 3. Користувач отримує відповідь чи оплачений чек (ціна записана у коді, або можна ввести з клавіатури). (звернути увагу на імена змінних

```
print ("Привіт")
price1 = 50 #price1=int(input())
price2 = 32 #price2=int(input())
if price1+ price2 > 99 :
    print ("Сума не достатня")
else:
    print ("Чек оплачений")
    print ("До побачення")
```

Приклад 4. Вивести на екран місяці відповідної попи року

```
s=input("Введіть пору року")
if s=="зима":
    print("грудень")
    print("січень")
    print("лютий")
elif s=="весна":
    print("березень")
    print("квітень")
    print("травень")
elif s=="літо":
    print("червень")
    print("липень")
    print("серпень")
else:
    print("вересень")
    print("жовтень ")
    print("листопад")
print ("До побачення")
```

Контрольні питання до глави 4

1. Що таке умовний оператор `if` у Python? Для чого він використовується?
2. Які оператори порівняння підтримує Python? Наведіть приклади.
3. Чим відрізняється оператор присвоєння = від оператора перевірки на рівність ==?
4. Що таке ланцюгові порівняння у Python? Наведіть приклад.
5. Які основні синтаксичні форми умовного оператора існують у Python?

6. У чому полягає відмінність між неповною та повною формою умовного оператора?
7. Як працює багатогілкова форма умовного оператора з `elif`?
8. Яка роль двокрапки `:` у синтаксисі умовних операторів?
9. Що відбувається, якщо умова у операторі `if` є хибною?
10. Як можна записати умовний оператор у одному рядку?

Практичні завдання до глави 4

1. Реалізуйте програму, яка за двома введеними числами перевіряє, чи можна знайти частку від ділення першого на друге.
2. Напишіть програму, яка за введеними координатами точки вознaчає чверть.
3. Складіть програму, яка визначає найбільше з трьох введених чисел.
4. Реалізуйте програму, яка перевіряє, чи введене число є парним або непарним.
5. Напишіть програму, яка визначає, чи введене число кратне 10.
6. Складіть програму, яка за введеним віком користувача виводить повідомлення: «дитина», «підліток», «дорослий», «пенсіонер».
7. Реалізуйте програму, яка перевіряє, чи введений рік є високосним.
8. Напишіть програму, яка за введеною оцінкою (від 1 до 12) виводить текстове пояснення («незадовільно», «задовільно», «добре», «відмінно»).